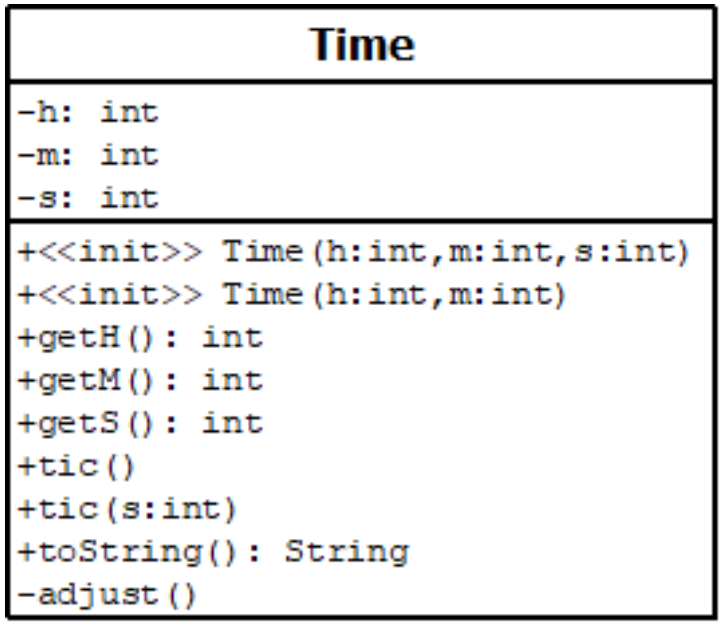
Mer om klasser, objekt och Java

this, static, ”immutable” objekt, enum

Klassen String

Inkapsling - försök själv (1) …

- Skriv en klass, Time, vars objekt representerar en tidpunkt i timmar, minuter och sekunder
- Lämpliga metoder? Datamedlemmar?
- Kontrakt?
  - En instans (objekt) av typen Time representerar en tidpunkt, h, m och s, där det alltid gäller att
    $0 \leq h < 24$ och
    $0 \leq m, s < 60$



Försök själv (1) …

- tic() och tic(int) ska räkna upp tiden, en respektive s sekunder
  - Metoderna har samma namn, exempel på överlagrade (overloaded) metoder
- Den privata hjälpmetoden adjust ska justera h, m och s så att de följer kontraktet
  - Metoden adjust anropas i allmetoder som kan ändra värde på h, m eller s, dvs. konstruktorer, tic, …
  - Metoden är privat eftersom den endast är för internt bruk – ska inte ingå i klassens publika gränssnitt
- Lösningsförslag finns på kurswebben

```java
public class Time {

    // private fields
    private int h, m, s;

    // constructors, overloaded
    public Time(int h, int m, int s) {
        this.h = h;
        this.m = m;
        this.s = s;
        adjust();
    }

    public Time(int h, int m) {
        this(h, m, 0);
    }

    // access methods
    public int getHours() {
        return h;
    }

    …

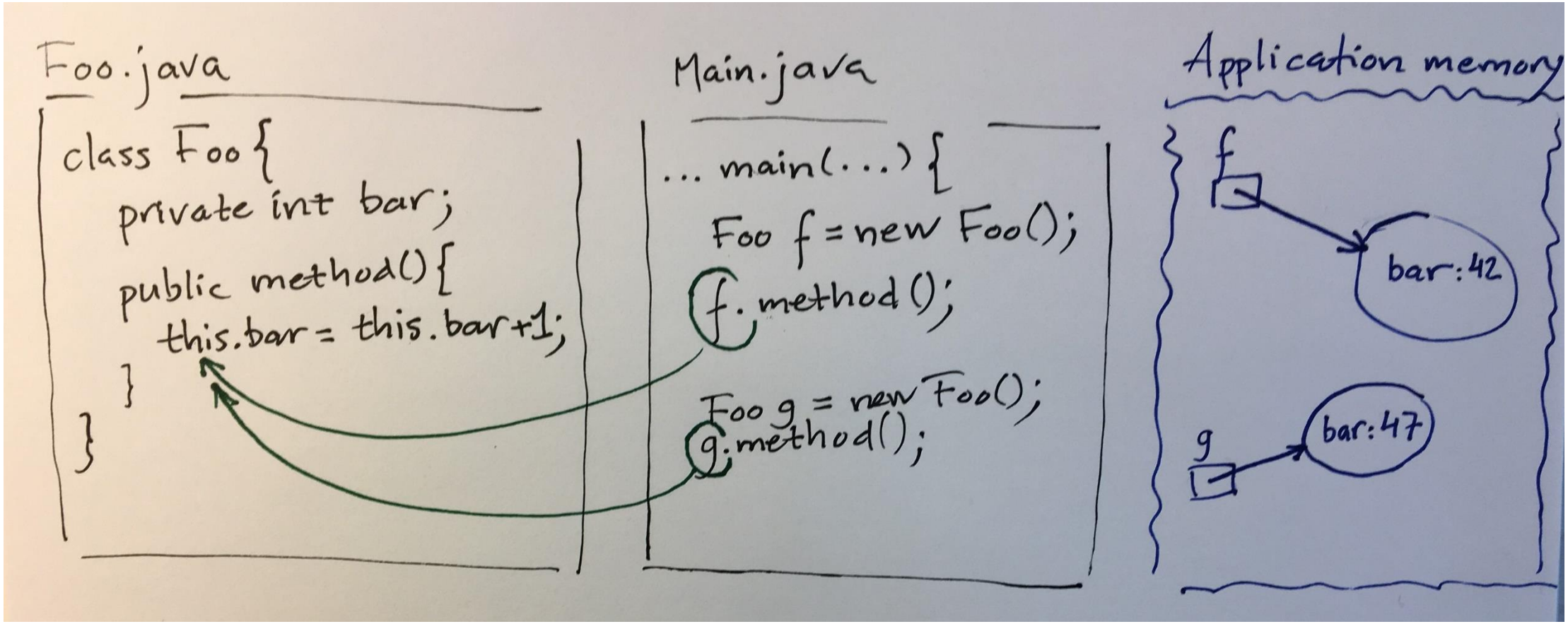
    public void tic(int s) {
        this.s += s;
        adjust();
    }

    public void tic() {
        this.tic(1);
    }

    // other methods
    @Override
    public String toString() {
        return String.format("%02d:%02d:%02d", h,m,s);
    }

    // private helper method, adjusting the values
    // of h, m and s.
    private void adjust() {
        m += s / 60;
        h += m / 60;
        // "%" calculates the remainder (modulus)
        s = s % 60;
        m = m % 60;
        h = h % 24;
    }
}
```

Nyckelordet this – så fungerar det …



Mall för Foo (klassen)

Skapa objekt av typen Foo

Tre sätt att använda nyckelordet this

- Nyckelordet ”this” kan användas i en metod för att referera till det objekt som metoden exekveras för, ”current object”

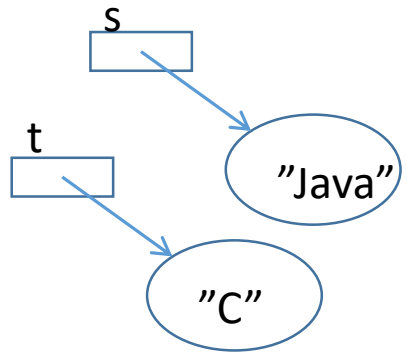
1. Vi kan, när vi skriver en klass, i en metod skilja på lokala variabler och datamedlemmar med samma namn
   - `this.balance = balance;`

2. Vi kan referera till det objekt som exekverar metoden, t.ex. för att skicka det som argument till en annan metod
   - ```
     public void printInfo() {
        System.out.println(this);
     }
     ```

3. Vi kan i en konstruktor anropa en annan konstruktor med this
   - ```
     public void Account(String name) {
        this(name, 0.0);
     }
     ```

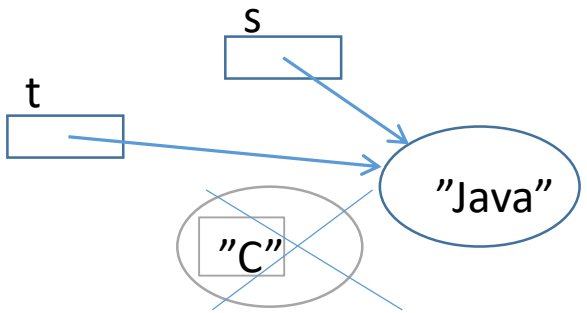
Data av klasstyp: referenser vs objekt

- ```
  String s = new String(”Java”);
  String t = new String(”C”);
  ```



- ”=”, tilldelning, kopierar alltid variabelns värde i Java, men i detta fall är värdet en referens, ”adress”

- ```
  t = s; // t and s are aliases
  ```



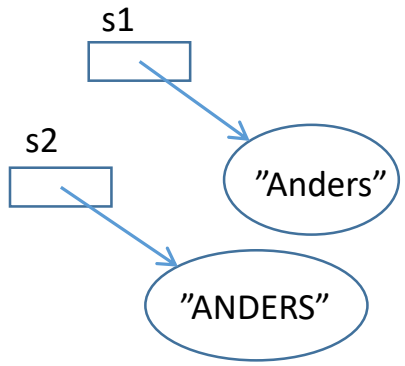
- Ingen referens till objektet ”C” – blir garbage collected dvs Javas virtuella maskin rensar objektet från minnet

- NB! Om en datamedlem är en referensvariabel till ett objekt måste vi tänka till om vi ska deklarera en accessmetod för denna

Klasser vars objekt är immutable

- Immutable = oföränderlig
- En klass vars objekt inte kan ändra tillstånd efter de skapats
- Exempel i Javas API: java.lang.String, java.time.LocalDate
  - Det går inte att ändra i ett String-objekt – skapa istället ett nytt
- Många metoder i klassen String som ”verkar” ändra i objektet returnerar i stället ett nytt objekt …

```
String s1 = new String(”Anders”);
String s2 = s1.toUppercase();
// creates a second object
```




```
String s3 = s1.concat(s2);
// creates a third object
// … the same as s3 = s1 + s2
```

En klass vars objekt är immutable

- Vi vill representera ett spelkort, med färg (suit) och valör (rank)
- Objektens tillstånd, färg och valör, ska inte kunna ändras efter att det skapats, dvs immutable
- Vi åstadkommer detta genom att låta …
  - datamedlemmarna vara privata och …
  - … inte deklarera mutator-metoder
- Det är också lämpligt att deklarera datamedlemmarna som final
- NB! I detta fall är datamedlemmarna av primitivt typ (int) – accessmetoderna lämnar ut en kopia av värdet

```
public class Card {

    private final int suit;
    private final int rank;

    public Card(int suit, int rank) {
        this.suit = suit;
        this.rank = rank;
    }

    public int getSuit() {
        return suit;
    }

    public int getRank() {
        return rank;
    }

    @Override
    public String toString() {
        String info = …
        return info;
    }
}
```

Klasser vars objekt är immutable

- Poänger med immutable:
  - Objekt kan återanvändas, t.ex. samma String-objekt i olika delar av koden
  - Inte ett problem att lämna ut en privat referens från ett objekt om det som refereras är av en typ som är immutable
  - Undviker problem i fler-trådade applikationer (mer om detta senare i kursen)
- NB! final för referensvariabler innebär att referensen inte kan ändras, men *hindrar inte att objektet som refereras kan ändras*
  - ```
    final Account a = new Account(…); // mutable class
    a.withdraw(100.0); // the state of the object is changed
    ```
  - ```
    Card c = new Card(HEARTS, ACE);
    // immutable class, object state cannot change
    ```

Försök själv (2)…

- Skriv en klass, Complex, som representerar ett komplext tal, med realdel och imaginärdel: a + bi
- Exempel på räkneregler:
  - Addition: (a + bi) + (c + di) = a+c + (b+d)i
  - Multiplikation: (a + bi) * (c + di) = (ac-bd) + (ad+bc)i
- Kontrakt:
  - Objekt av typen Complex ska vara immutable
  - *Addition respektive multiplikation ska generera ett nytt objekt*, resultatet
  - c1.add(Complex other), c1.add(double re)

Försök själv (2)…

- Datamedlemmarna, re och im, deklareras final
- Lämpliga steg:
  - Skriv konstruktorer och en toString-metod – skapa objekt i test-main
  - Skriv accessmetoder för data (getters) – testa
  - Lägg till add(Complex other) – testa

Enum – varför?

- I vår tidigare version av Card använde vi den primitiva typen int för att representera suit och rank …
- … men både suit och rank ska egentligen endast tillåta värdena 1-4 respektive 1-13
- I vår tidigare version var det möjligt att skapa
  - `new Card(0, 42)`
  - new Card(12, 4) – här har vi förväxlat suit och rank i konstruktorn
- Vi kan använda oss av exceptions, men felen upptäcks inte förrän koden exekveras
- Vi vill att denna typ av fel ska upptäcks redan vid kompileringen …
  - … innan applikationen exekveras

```
public class Card {

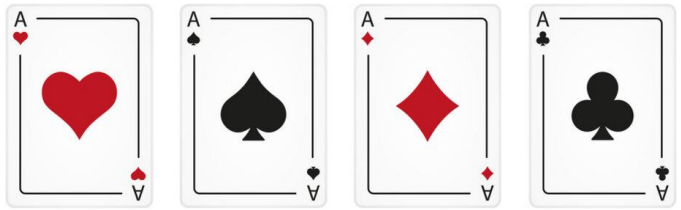
    private final int suit;
    private final int rank;

    // Constructor – dont do this
    public Card(int suit, int rank) {
        // Don't allow creation of illegal cards
        if (suit >= 1 && suit <= 4 &&
            rank >= 1 && rank <= 13)
               throw new IllegalArgumentException(…);

        this.suit = suit;
        this.rank = rank;
    }

    …
```

Enum



- Enumeration, uppräkningsbar
- Använd enum då du behöver en typ med ett begränsat antal fördefinierade värden, t.ex. för tillåtna färger (suit) i kortspel
- Deklarera i en separat fil, Suit.java, eller inuti en klass
- `public enum Suit { HEARTS, SPADES, DIAMONDS, CLUBS; }`
- Används så här:
  - `Suit s = Suit.CLUBS;`
  - `if(s == Suit.CLUBS) { …`
- `public enum Rank { ACE, TWO, …, KING; }`

Klassen Card med enums

```
public class Card {

    private final Rank rank;
    private final Suit suit;

    public Card(Rank rank, Suit suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public Rank getRank() {
        return rank;
    }

    public Suit getSuit() {
        return suit;
    }

    public int getSuitValue() {
        return suit.ordinal();
    }
```

```
    …

    @Override
    public String toString() {
        String info = rank + " of " + suit;
        return info;
    }
}
```

Klassen Card med enums – skapa objekt

Klassen, mallen

```
public class Card {

    private final Rank rank;
    private final Suit suit;

    public Card(Rank rank, Suit suit) {
        this.rank = rank;
        this.suit = suit;
    }

    …
```

Skapa objekt med enum-värden

```
public class CardMain {
  public static void main(…) {

    Card c1 = new Card(Rank.ACE, Suit.HEARTS);

    Card c1 = new Card(Rank.ESS, Suit.HEARTS);

    Card c1 = new Card(Suit.HEARTS, Rank.ACE);
  }
}
```

- Det är nu omöjligt att
  - förväxla rank och suit, då de är av olika typ
  - ange ogiltiga värden, som rank = 100 eller rank = JOKER
- Enum-metoden ordinal returnerar en int, ordningsnumret för värdet
- Skriv koden så att kompilator + IDE tidigt kan varna för fel, använd
  - Inkapsling
  - final
  - Enum

Enum, egentligen en mycket speciell klass …

- Enum är egentligen en klasstyp …
  - … som kan ha datamedlemmar, konstruktorer och metoder
  - … där enum-värderna egentligen är klasskonstanter (static final)

```
public enum Rank {

    ACE(1), TWO(2), …, KING(13);

    public int getValue() {
        return value;
    }

    private final int value;

    private Rank(int value) {
        this.value = value;
    }
}
```

Metod eller funktion?

- Vad är en (matematisk) funktion?
  - Indata => utdata, t.ex. $f(x) = x^2$, $f(3) = 9$
  - Ett och samma indata till en funktion ger alltid samma utdata …
  - … oavsett när eller hur många gånger funktionen anropas
  - Normalt är det så här funktioner i språket C fungerar

- Men i ett objektorienterat språk anropas metoder på ett objekt, en instans …
- … och kan påverka objektets tillstånd (state)
  - Anropet acc.withdraw(1000) kan ge olika resultat/tillstånd beroende på objektets tidigare tillstånd
- Sidoeffekt, i OO-kontext, är …
  - … när något sker/ändras vid ett metodanrop förutom det vi ser i form av utdata …
  - … t.ex. en ändring av objektets tillstånd, en utskrift, …
- Metoder, egenligen instansmetoder, är alltså inte ”rena” funktioner …
  - … och därför har man valt ett annat namn än ”funktion” för OO-metoder

Kan vi ha ”vanliga” funktioner i OO?
Ja - klassdata och klassmetoder - static

- Tidigare när vi deklarerat en datamedlem eller en metod …
  - … har varje objekt en egen uppsättning av datamedlemmen (instansvariabel)
  - … måste metoden anropas på ett specifikt objekt via en (instans-) metod
- Vi kan i objektorienterad programmering även ha datamedlemmar och funktioner som inte är kopplade till objekt/instanser …
  - … men till en klass
- En datamedlem som deklareras static är gemensam för alla objekt av denna klass typ, s.k. klassdata, klassvariabel
  - Klassdata initieras vid deklarationen eller i ett static-block …
  - … inte via konstruktorer, dessa är till för initiering av instansvariabler
- En metod som deklareras static, s.k. klassmetod, anropas utan en objektreferens
  - Kan inte accessa instansvariabler, som ju är kopplade till specifika objekt
  - Mer som en funktion, t.ex. i C

Klassdata och klassmetoder - static

- static/klass-metoder är lämpliga
  - när vi vill definiera funktionalitet som inte representerar något objekts tillstånd
- Exempel
  - En klass, Math, med matematisk funktionalitet och konstanter
  - Notera att vi måste skriva dessa klassmetoder/funktioner i en klass …
  - … trots att vi i detta fall inte behöver/bör skapa objekt

```
public class Math {

    public static final double PI = 3.14159265359;
    public static final double E = 2.71828182845;

    public static double abs(double value) {
        if(value < 0) return -value;
        return value;
    }

    public static double pow(double base, int exp) {
        if(exp == 0) return 1;
        if(exp > 0) {
            double res = base;
            for(int i = 2; i <= exp; i++) {
                res *= base;
            }
            return res;
        }
        …
    }
```

Klassdata och klassmetoder - static

- Vi anropar klass-/static-metoder genom att ange klassnamnet
- Motsvarande för klass-/static-data
- I detta fall skapar vi inte något ”Math-objekt”
  - Vi kan göra det omöjligt att skapa objekt genom att deklarera en privat konstruktor

```
// main …
double radius = 5.0;
double area = radius * radius * Math.PI;
…

double a1 = Math.abs(5.0);
…
double a2 = Math.abs(-5.0);
…

double base = 2.5;
int exp = 3;
double res = Math.pow(base, exp);
…
```

Klassdata och klassmetoder i en ”vanlig” klass

- Vi kan ha både instansvariabler/instansmetoder respektive klassdata/klassmetoder i en klass
- Exempel: Om vi vill ha en ränta som är *gemensam för alla A*ccount-objekt kan vi deklarera en static datamedlem (klassdata) för detta, med tillhörande static-metoder
  - Räntan finns då i en enda upplaga, oavsett hur många Account-objekt som skapas och …
  - … data är gemensam för alla Account-objekt

Klassdata, static, i Account

```
public class Account4 {

    // instance fields
    private double balance;
    private final String name;

    private static double interest = 0.25; // static member, i.e. class data

    public Account4(String name, double balance) {
        this.name = name;
        this.balance = balance;
    }

    // instance methods
    public double getBalance() { return balance;  }
    …

    // static methods; can only reference static members
    public static double getInterest() { return interest; }

    public static void setInterest(double newInterest) { interest = newInterest;
```

Klassdata, static, i Account

- Anrop, t.ex. i main

```
Account4 anders = new Account4("Anders", 1000);
Account4 ibrahim = new Account4("Ibrahim", 100000.0);


// only affects the specified object
ibrahim.withdraw(50000.0);


// interest is static, affects all objects
Account4.setInterest(0.50);
```